AI-Driven Software Development Methodology

A Practitioner's Guide to Disciplined, Traceable, LLM-Assisted Engineering

Version: 1.0
Date: 2026-04-21
Author: Florin Ciontu

Table of Contents

1. Overview
2. Core Principles
3. The Artifact Hierarchy
4. The AI Operating Contract (CLAUDE.md)
5. Project Structure and File Layout
6. The Planning Layer
7. Story and Task Decomposition
8. The Execution Flow — Step by Step
9. Self-Validation Technique
10. Adversarial Validation Technique
11. Testing Requirements and Standards
12. Traceability Requirements
13. The Frozen Baseline Protocol
14. The Baseline Change Protocol
15. Evidence-Based Completion
16. The Implementation Log
17. Memory and Context Management
18. Common Failure Modes and How to Avoid Them
19. Worked Example — End to End

Annexes

- Annex A — CLAUDE.md Example (Full)
- Annex B — Implementation Plan Structure (Full Example)
- Annex C — Story Specification Example (Full)
- Annex D — Implementation Log Entry Examples
- Annex E — Self-Validation Prompt Examples
- Annex F — Adversarial Validation Prompt Examples
- Annex G — Code Traceability Examples
- Annex H — RFC and ADR Templates

---

1. Overview

This document describes a methodology for building production software systems using AI coding assistants (specifically large language model agents such as Claude Code) as the primary implementation vehicle, with a human as architect, reviewer, and final decision-maker.

The methodology solves a fundamental problem: AI coding agents are capable of writing large amounts of code very quickly, but without discipline they produce code that is:

- Inconsistent across sessions (the AI forgets what it decided)
- Untraceable (no link between a line of code and the decision that required it)
- Untestable (tests are added after the fact, if at all)
- Architecturally drifting (each session slightly reinterprets the design)
- Silently incorrect (the AI changes architecture to make code work, without flagging the deviation)

This methodology imposes structure that eliminates these failure modes while preserving the speed advantage of AI-assisted development. The key insight is: **the AI is an extremely capable implementer, but a poor architect and an unreliable guardian of prior decisions**. The human provides all architectural direction; the AI executes it within a tightly constrained operating envelope.

What this methodology produces

- A fully traceable codebase where every file, function, and test links back to a specific requirement
- An implementation log that serves as a complete audit trail of every decision and change
- A frozen baseline (PRD, RFCs, ADRs) that cannot be silently modified
- A test suite that grows in lockstep with implementation
- A replayable history: any future developer (human or AI) can reconstruct the full reasoning from the artifacts alone

What this methodology is not

This is not a waterfall process. Stories are decomposed and validated just-in-time. The architecture emerges from validated RFCs, not upfront big-design. The frozen baseline captures decisions, not code — code is the output of those decisions.

---

2. Core Principles

P1 — Human as Architect, AI as Implementer

The human defines what to build and why. The AI builds it. The AI never makes architectural decisions; it may identify options and trade-offs, but the human selects. When the AI encounters an ambiguity it cannot resolve within the existing spec, it stops and asks — it never silently resolves architectural questions in favor of implementation convenience.

P2 — Nothing Untracked

Every file, every function, every test is linked to a specific Epic/Story/Task. Code that cannot be traced to a work item should not exist. The AI is instructed to refuse to write "quick fixes" or "improvements" outside the planned structure.

P3 — Frozen Baseline

The product requirements, accepted RFCs, and accepted ADRs form a frozen baseline. Once accepted, they cannot be silently changed during implementation. If implementation reveals a conflict with the baseline, a formal change process is triggered — implementation stops until the conflict is resolved.

P4 — Evidence Before Completion

Work is not marked done until evidence exists: tests pass, documentation is updated, traceability is recorded. The AI cannot mark a task complete by assertion — it must show the evidence (test output, log entries, passing CI).

P5 — Self-Validation Before Execution

Before writing any code, the AI is required to validate its plan against the spec, identify potential conflicts, and surface assumptions. This is a structured internal review, not a narrative summary. It catches bugs before they are committed.

P6 — Tests Are Written During Implementation

Tests are not written after. Every new function has a corresponding test written in the same session. The AI is prohibited from creating "temporary code without tests."

P7 — Incremental Ordered Execution

Work proceeds epic by epic, story by story, task by task. Jumping ahead is prohibited. This enforces dependency order and prevents the AI from building on foundations that haven't been validated yet.

P8 — Explicit Provenance

Data written to any store (database, file, cache) includes provenance metadata: who wrote it, when, from which source, with what confidence. This applies to both deterministic and AI-generated outputs.

---

3. The Artifact Hierarchy

The methodology uses a layered artifact hierarchy. Each layer constrains the layers below it.

```
PRD (Product Requirements Document)
  └── RFC (Request for Comments — accepted architecture decisions)
        └── ADR (Architecture Decision Records — implementation-level decisions)
              └── EPIC (large feature area, 4–12 weeks)
                    └── STORY (user-facing capability, 1–5 days)
                          └── TASK (implementer-level unit of work, 2–8 hours)
```

PRD

The single source of truth for what the product does and why. Written before any code. Defines user personas, key workflows, constraints, and non-goals. The PRD does not specify how — only what.

RFC (Request for Comments)

Each RFC addresses a specific architectural question raised during planning. An RFC is a structured document with: problem statement, options considered, decision, rationale, and consequences. RFCs are numbered sequentially and are immutable once accepted. If a subsequent RFC contradicts a prior one, the prior one is formally amended with a reference to the superseding RFC.

RFC covers: system design choices, data models, API contracts, integration patterns, provider interfaces, error handling strategies.

ADR (Architecture Decision Record)

More granular than RFCs. ADRs capture implementation-level decisions that don't warrant a full RFC but need to be recorded. Example: "ADR-009: use Decimal(10,2) for monetary values, not float."

EPIC

A large body of work representing a coherent product capability. An Epic has: a purpose statement, a list of stories, an integration checkpoint (a testable milestone), and a deployment milestone. Epics are numbered (EPIC-001, EPIC-002, ...).

STORY

A single user-facing or system-level capability that can be built and tested in one session. A Story has: a user story statement ("As a [role] I want [capability] so that [value]"), acceptance criteria, scope in/out, task list, dependencies, and a status field (planned → in_progress → in_review → done).

TASK

A discrete unit of implementation work. A Task maps to specific files and specific code. Tasks are the unit at which the AI operates — each session begins by identifying the current task, executing it, and confirming completion with evidence.

---

4. The AI Operating Contract (CLAUDE.md)

The most important single artifact in this methodology is `CLAUDE.md` — a file placed in the project root that the AI reads at the start of every session. It is the operating contract between the human and the AI.

`CLAUDE.md` contains:

1. **Project overview** — one paragraph on what is being built
2. **Frozen baseline reference** — explicit list of frozen documents with file paths
3. **Mandatory pre-work rules** — what the AI must do before starting any implementation (consult implementation plan, verify story status, check dependencies)
4. **During-implementation rules** — ordering constraints, tracking requirements, baseline change triggers
5. **Post-step rules** — what must be updated after every meaningful change (implementation log, story status, test evidence)
6. **Completion checklists** — for task, story, and epic completion
7. **Testing standards** — file locations, naming conventions, coverage expectations
8. **Traceability format** — the exact comment format required in every source file
9. **Prohibited actions** — an explicit list of things the AI must never do
10. **Required discipline statement** — a plain-language assertion of the operating mode

The critical design principle of `CLAUDE.md` is that it is **instruction, not guidance**. Every rule is mandatory. The document uses language like "MUST", "FORBIDDEN", "REQUIRED", not "should" or "consider". This matters because LLMs are trained to be helpful and will take shortcuts if rules are expressed as suggestions.

> *See **Annex A** for a complete CLAUDE.md example.*

---

5. Project Structure and File Layout

```
project-root/
├── CLAUDE.md                          # AI operating contract (read every session)
├── docs/
│   ├── prd/
│   │   └── PRD.md                     # Frozen product requirements
│   └── architecture/
│       ├── IMPLEMENTATION-PLAN-V1.md  # Master plan with status tracking
│       ├── IMPLEMENTATION-LOG.md      # Append-only audit log
│       ├── RFC-001-*.md               # Accepted RFCs (immutable once accepted)
│       ├── RFC-002-*.md
│       └── ADR-001-*.md               # Architecture Decision Records
├── stories/
│   ├── README.md                      # Stories index
│   ├── epics/
│   │   ├── EPIC-001-*.md
│   │   └── EPIC-002-*.md
│   └── tasks/
│       ├── EPIC-001-*/
│       │   ├── STORY-001-*.md
│       │   └── STORY-002-*.md
│       └── EPIC-002-*/
│           └── STORY-010-*.md
├── src/                               # Application source code
│   └── modules/
│       └── [module-name]/
│           ├── [feature].service.ts   # Each file has traceability comments
│           └── [feature].test.ts
├── tests/
│   ├── unit/                          # Unit tests — mirror src/ structure
│   ├── integration/                   # Integration tests
│   └── e2e/                           # End-to-end tests
└── prisma/                            # (if using Prisma ORM)
    ├── schema.prisma
    └── migrations/
```

Key structural rules

- The `docs/architecture/` directory is the authoritative planning layer. Source code is subordinate to it.
- `IMPLEMENTATION-LOG.md` is append-only — entries are never edited or deleted, only added.
- Story files are the primary source of truth for what each story requires. They are written before implementation begins, not derived from it.
- Test files mirror the source directory structure. A test for `src/modules/foo/bar.service.ts` lives at `tests/unit/foo/bar.service.test.ts`.

---

6. The Planning Layer

The Implementation Plan

The Implementation Plan (`IMPLEMENTATION-PLAN-V1.md`) is the master tracking document. It contains:

- **Status summary** — current phase, active epic, active story, overall progress
- **Status model** — definition of each status value (planned, validated, in_progress, done)
- **Epic list** — one section per epic, with story list and status
- **Story list** — one entry per story with: status, dependencies, task summary, spec link
- **Integration checkpoints** — testable milestones between epics

The Implementation Plan is updated continuously as work progresses. It is the first document the AI reads at the start of every session ("consult the implementation plan before starting any work").

Updating the Plan

The AI updates the plan:

- When a task is completed → mark task done in the story entry
- When a story is completed → mark story done, update status field
- When an epic is completed → mark epic done, unlock next epic
- When a blocker is found → add blocker note immediately

The human reviews the plan at the start of each working session to confirm the AI's tracking is accurate.

Story Validation

Before a story can be implemented, it must be **validated**:

1. Story spec is written (user story, acceptance criteria, tasks, dependencies)
2. Reviewed against PRD — does this story serve the PRD's requirements?
3. Reviewed against accepted RFCs — does this story follow accepted architecture?
4. Dependencies confirmed as satisfied (prior stories done, APIs available, etc.)
5. Status set to `ready`

A story with status `planned` cannot be implemented. Only `ready` stories proceed.

---

7. Story and Task Decomposition

Story Specification

A story specification is written before implementation. It follows a fixed structure:

```
# STORY-NNN — [Title]

## Epic
EPIC-NNN — [Epic name]

## Status
planned | ready | in_progress | done

## Purpose
[Why this story exists — business or technical motivation]

## Story
As a [role],
I want [capability],
so that [value delivered].
```

```
## Scope In
### Task 1 — [Task name]
[Specific files to create/modify, logic to implement, interfaces to satisfy]

### Task 2 — [Task name]
[...]

## Scope Out
- [Explicit list of things this story does NOT do]

## Acceptance Criteria
- [ ] [Testable criterion 1]
- [ ] [Testable criterion 2]

## Dependencies
- STORY-NNN (must be done first — reason)
- External: [API availability, schema migration, etc.]
```

Task Granularity

Tasks are sized to be completable in one AI session (typically 1–4 hours of elapsed work). A task that touches more than 3–4 files is probably too large and should be split. The test for correct granularity: can the task be described in one sentence that specifies exactly which file(s) to create or modify and what they must do?

Good task: "Create `detectHoldingCompany()` in `src/modules/classification/detectors/holding-company.detector.ts` — takes `StockMetadata`, calls LLM with `holding-company.md` prompt, returns `{ flag: boolean | null, confidence: number, provenance: ProvenanceEntry }`"

Bad task: "Implement holding company detection" (too vague — doesn't specify file, interface, or output shape)

Pre-Task Checklist

Before executing any task, the AI confirms:

- ☐ Implementation plan consulted — story status is `ready`
- ☐ All dependencies satisfied
- ☐ RFC/ADR references identified for this task
- ☐ Interface contracts understood (input/output types)
- ☐ Test file location determined
- ☐ No baseline conflicts identified

---

8. The Execution Flow — Step by Step

This is the complete execution sequence for a single story, from start to completion.

```
┌──────────────────────────────────────────────────────────────┐
│  SESSION START                                               │
```

```
│  1. Read CLAUDE.md                                          │
│  2. Read IMPLEMENTATION-PLAN-V1.md                          │
│  3. Identify current active story                           │
│  4. Confirm story status = ready                            │
│  5. Check for open blockers                                 │
└─────────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌─────────────────────────────────────────────────────────────┐
│  PRE-TASK: SELF-VALIDATION                                  │
│  6. Read the story spec in full                             │
│  7. Read all referenced RFC/ADR sections                    │
│  8. Identify: files to create/modify, interfaces to         │
│     satisfy, tests to write                                 │
│  9. Run self-validation checklist (see Section 9)           │
│  10. Surface any conflicts or ambiguities to human           │
│  11. Human approves plan or adjusts                          │
└─────────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌─────────────────────────────────────────────────────────────┐
│  IMPLEMENTATION                                             │
│  12. Write code for Task N                                  │
│  13. Write tests for Task N (in same session)               │
│  14. Run tests — confirm passing                            │
│  15. Update story file: mark Task N done                    │
│  16. Append entry to IMPLEMENTATION-LOG.md                  │
│  17. Repeat 12–16 for each task in the story                │
└─────────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌─────────────────────────────────────────────────────────────┐
│  STORY COMPLETION                                           │
│  18. Run full test suite — confirm no regressions           │
│  19. Verify all acceptance criteria met                     │
│  20. Update story status → done                             │
│  21. Update IMPLEMENTATION-PLAN-V1.md                       │
│  22. Append story completion entry to log                   │
│  23. Commit with message: [EPIC-NNN/STORY-NNN] ...          │
└─────────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
              Next story or epic
```

---

9. Self-Validation Technique

Self-validation is a structured pre-implementation review that the AI performs on its own plan before writing any code. It is not a narrative summary — it is a checklist-driven audit.

When to run self-validation

- Before implementing any story

- Before any task that touches shared interfaces or external systems
- When a story spec references multiple RFCs or ADRs
- Whenever the AI is uncertain about any aspect of the design

The self-validation checklist

The AI works through these questions explicitly, writing its answers:

Spec consistency

- Does my implementation plan match the story spec exactly, or am I interpolating?
- Have I read the full story spec, not just the task descriptions?
- Are there any scope-out items I might be tempted to implement anyway?

Interface contracts

- What are the exact input and output types for each function I'm about to write?
- Do these types match what upstream callers will provide and downstream consumers expect?
- Have I checked the relevant RFC for the canonical interface definition?

Dependency order

- Does this task depend on anything that isn't done yet?
- Am I about to write code that calls a function that doesn't exist yet?

Baseline alignment

- Does my plan align with all referenced RFCs and ADRs?
- Am I about to make an architectural decision that should be in an RFC but isn't?

Test coverage

- What are the exact test cases I'll write for this task?
- Do my test cases cover: happy path, error paths, edge cases (null inputs, boundary values)?
- Are there regression risks in adjacent code?

Schema and data

- If I'm writing to a database, do the column types match the schema?
- Have I checked for any pending migrations that would affect this code?

Known risks

- What is the most likely way this implementation will be wrong?
- What assumption am I making that could be false?

Self-validation output format

The AI produces a structured pre-flight note:

```
SELF-VALIDATION — STORY-040 / TASK-040-002

Spec match: ✓ — implementing exactly what Task 2 specifies
Interface: EnrichmentScoresInput → CombinedEnrichmentOutput (matches RFC-007 §4.2)
Dependencies: PromptLoader (STORY-034 ✓), LLMProvider (STORY-034 ✓), schema columns
(STORY-039 ✓)
Baseline: RFC-007 confidence gating, half-integer rounding — both in spec
Tests: 5 cases identified — all outputs above threshold, scores_confidence below
threshold,
```

```
       one flag below threshold, rounding verification (3.7→3.5, 3.8→4.0), LLM error
path
Schema risk: DECIMAL(3,2) supports 1.0–5.0 in 0.5 steps — confirmed ✓
Known risk: nested function variable shadowing if confidence destructuring is misused
— will use
            explicit destructuring: const { flag, confidence } = result[llmKey]

Proceeding.
```

If any item in the self-validation produces a red flag, the AI stops and raises it to the human before proceeding.

> *See **Annex E** for full self-validation prompt examples.*

---

10. Adversarial Validation Technique

Adversarial validation goes beyond self-validation. Where self-validation asks "is my plan consistent with the spec?", adversarial validation asks "how could my plan be wrong, and what would break if it is?"

The AI deliberately tries to break its own plan — finding counterexamples, edge cases, and hidden assumptions.

When to use adversarial validation

- Before implementing any component that handles external data (API responses, user input, database reads)
- Before implementing any shared interface that multiple callers will use
- When the spec has implicit assumptions (e.g., "confidence is between 0 and 1 — but what if the LLM returns 1.2?")
- When the implementation involves concurrency, ordering dependencies, or stateful operations
- After finding a bug — to identify other bugs of the same class

The adversarial validation process

Step 1 — Identify the assumptions

List every assumption the implementation makes. Be exhaustive.

Example for a confidence-gated LLM call:

- The LLM always returns a JSON response (assumption: API doesn't time out or return malformed JSON)
- `confidence` is always a float between 0 and 1 (assumption: LLM respects schema constraints)
- The prompt file always exists at the expected path (assumption: deployment includes prompt files)
- `scores_confidence` covers all 6 scores with a single value (design choice — is this correct?)
- `result[llmKey]` always has `flag` and `confidence` fields (assumption: schema validation works)

Step 2 — Attack each assumption

For each assumption, ask: "What happens if this is false?" and "Is the code correct in that case?"

- LLM times out → code must catch the exception and return error provenance, not throw
- Confidence is 1.2 → gate condition `>= 0.60` still fires correctly — OK
- Prompt file missing → fs.readFileSync throws — is it caught? Yes, outer try/catch handles it
- Schema validation fails → LLM returns partial object — destructuring `result[llmKey]` will throw undefined access → caught by outer try/catch

Step 3 — Write tests for the adversarial cases

Any assumption that could realistically fail becomes a test case:

- LLM API error → `makeErrorProvider()` test
- scores_confidence below threshold → separate test case
- One flag below threshold → tests isolation of per-flag gating

Step 4 — Review the result

The adversarial pass either: a) Finds real bugs → fix them before writing the production code b) Confirms the design is robust → proceed with higher confidence

The adversarial mindset

The most valuable adversarial question is: **"What would a junior developer get wrong here?"**

- Off-by-one in threshold comparison (`>` vs `>=`)
- Forgetting null vs undefined distinction
- Mutating a shared object
- Not handling the case where an async operation resolves to null
- Assuming a database column is non-null when the schema allows null

> *See **Annex F** for full adversarial validation prompt examples.*

---

11. Testing Requirements and Standards

The non-negotiable rule

Every new function requires tests written in the same session. There are no exceptions. The AI is explicitly prohibited from writing "temporary code without tests."

Test file locations

```
tests/unit/          Unit tests — mock all external dependencies
tests/integration/   Integration tests — real database, real adapters
tests/e2e/           End-to-end tests — full stack, browser or API
```

Naming convention

```
describe('EPIC-003.1/STORY-040: detectCombinedEnrichment', () => {
  it('all outputs above threshold → all 3 flags written, all 6 scores written', ...)
  it('scores_confidence below threshold → all 6 scores empty, flags still written',
...)
})
```

The describe block includes the Epic/Story reference. The `it` block describes the exact condition and expected outcome in plain language. This means the test suite is a readable specification of the system's behaviour.

Coverage requirements

- Unit tests: >80% coverage for all new code
- Integration tests: all critical paths covered (database writes, external API calls)
- E2E tests: all user-facing workflows

Mock discipline

- Unit tests mock all external dependencies (database, HTTP, file system)
- Integration tests do NOT mock the database — they use a real test database
- The reason: mock/prod divergence has caused production failures (a schema mismatch that passing unit tests didn't catch)

Test structure — the Arrange/Act/Assert pattern

```
it('one flag below confidence threshold → that flag null, others written', async ()
=> {
  // Arrange
  const provider = makeMockProvider({ holdingConf: 0.40, cyclicalityConf: 0.90,
binaryConf: 0.90 });

  // Act
  const result = await detectCombinedEnrichment(BASE_INPUT, provider, new
PromptLoader());

  // Assert
  expect(result.flags['holding_company_flag']).toBeNull();
  expect(result.provenance['holding_company_flag']?.null_decision).toBe(true);
  expect(result.provenance['holding_company_flag']?.confidence).toBe(0.40);
  expect(result.flags['cyclicality_flag']).toBe(false);      // not affected
  expect(result.flags['binary_flag']).toBe(false);           // not affected
});
```

Regression testing

When a bug is fixed, a regression test is added that would have caught the bug. The test is added before the fix, confirmed to fail, then the fix is applied and the test is confirmed to pass.

---

12. Traceability Requirements

Every source file must include traceability comments at the top linking it to its epic, story, and task. There are no exceptions.

```
// EPIC-003.1: Classification LLM Enrichment
// STORY-040: E1–E6 Qualitative Enrichment Scores + Combined Enrichment Prompt
// TASK-040-002: EnrichmentScoresDetector — single combined LLM call returning 3
flags + 6 scores
```

Why traceability matters

When a bug is found 6 months later, the traceability comment is the entry point into the full decision chain:

- Task comment → story spec → acceptance criteria → RFC/ADR → baseline decision

Without traceability, the only context is the code itself. With it, the full reasoning is recoverable.

Commit message format

```
[EPIC-003.1/STORY-040/TASK-040-002] Implement detectCombinedEnrichment detector
```

Every commit references epic, story, and task. Commits without references are not allowed.

In-line traceability for non-obvious decisions

When a piece of code exists because of a specific decision or constraint that isn't obvious from the code itself, a comment explains why:

```
// BC-035-001: FMP /profile endpoint does not return SIC code in stable tier;
// sicCode is always null in production — holding_company pre-filter never fires.
sicCode: null,
```

This is not documenting *what* the code does (the code does that itself) — it documents *why* a seemingly wrong value (`null`) is intentional and links it to a named known constraint.

---

13. The Frozen Baseline Protocol

The frozen baseline consists of:

- The PRD
- All accepted RFCs (numbered RFC-001, RFC-002, ...)
- All accepted ADRs (numbered ADR-001, ADR-002, ...)

Once accepted, these documents are **immutable**. The AI cannot modify them during implementation. If the AI believes a baseline document needs to change, it must:

1. Stop implementation immediately
2. Log a `Baseline Impact: YES` entry in the implementation log
3. Describe the conflict explicitly: what the baseline says, what reality requires, what the proposed change is
4. Present the conflict to the human for decision
5. Wait for human approval before proceeding
6. If approved, formally amend the baseline document with a revision note and date

This protocol exists because AI agents have a strong tendency to "make things work" — silently adjusting architecture to resolve implementation difficulties. This produces working code that violates the original design intent, which only becomes visible much later when the downstream consequences appear.

Example of a baseline conflict

> ***STOP — Baseline Impact Identified***
>
> *Current baseline (RFC-004 §2.1) states: "The FMP `/stable/profile` endpoint returns `sicCode` for all stocks."*
>
> *Discovered during STORY-035 implementation: The `/stable/profile` endpoint does **not** return `sic_code` for any stock in the current FMP plan tier.*
>
> ***Proposed resolution:** Record as known constraint BC-035-001. Amend RFC-004 to acknowledge this gap. Pre-filter for `holding_company_flag` (which relies on SIC codes) always returns null in production — LLM must assess this flag for all stocks.*
>
> ***This changes:** The `needs_llm` decision logic (always true for holding_company_flag) and the architecture commentary in STORY-038.*
>
> ***Awaiting human approval before proceeding.***

---

14. The Baseline Change Protocol

When a baseline change is approved:

1. The relevant RFC or ADR is updated with a revision block:

```
---
**Amendment:** 2026-04-21 — BC-035-001 confirmed. FMP stable profile does not expose
SIC codes.
Pre-filter for holding_company_flag always returns null in production.
See implementation log entry 2026-04-21.
---
```

2. The implementation log records the baseline change with full context
3. The implementation plan is updated to reflect any scope changes
4. The implementation resumes from the point it was stopped

This creates a paper trail that explains why the codebase diverges from the original RFC in certain places — without it, future maintainers would see code that contradicts the RFC and have no way to understand why.

---

15. Evidence-Based Completion

"Done" is not a claim — it is a demonstrated state. The AI cannot mark work done without evidence.

Task completion evidence

- [ ] Implementation complete — specific files created/modified
- [ ] Tests written and passing — test file path and pass count recorded
- [ ] Traceability comments present in all new files
- [ ] Implementation log updated
- [ ] Task status updated to `done`

Story completion evidence

- ☐ All tasks done
- ☐ All acceptance criteria checked off
- ☐ Full test suite passes — suite name and pass count recorded
- ☐ No new TypeScript errors introduced (verified with `tsc --noEmit`)
- ☐ Story status updated to `done`
- ☐ Implementation log entry for story completion

Epic completion evidence

- ☐ All stories done
- ☐ Integration checkpoint passed
- ☐ Deployment milestone confirmed (if applicable)
- ☐ Implementation log entry for epic completion
- ☐ Epic status updated to `done`

What "tests passing" means

Tests are run with the actual test runner, not assumed to pass. The AI runs `npx jest tests/unit/ --no-coverage` (or equivalent) and records the exact output: N tests, N suites, N passing. If any test fails, that failure must be resolved before the task can be marked done.

---

16. The Implementation Log

The implementation log (`IMPLEMENTATION-LOG.md`) is an append-only audit trail of every meaningful change. It is the most important single source of truth for understanding the history of the project.

Log entry structure

Every log entry has these mandatory fields:

```
## [DATE] — [EPIC/STORY/TASK]: [One-line description]

**Epic:** EPIC-NNN — [name]
**Story:** STORY-NNN — [name]
**Task:** TASK-NNN-NNN (or "multiple" for story-level entries)

**Action:** [What was done — one paragraph]

**Self-validation findings:** [What the pre-flight check found — key decisions made]

**Files Changed:**
- `path/to/file.ts` (created|modified|deleted) — [what changed and why]

**Tests Added/Updated:**
- `tests/unit/path/test.ts` — [N tests: description of what they cover]

**Result/Status:** [DONE □ | BLOCKED □ | IN_PROGRESS] — [evidence: N/N tests passing]

**Blockers/Issues:** [None | description of any problem encountered]
```

```
**Baseline Impact:** YES/NO [if YES: describe what changed and why]

**Next Action:** [What the next task or story is]
```

Why the log matters

The log serves multiple purposes:

1. **Continuity across sessions** — the AI reads the last log entry at the start of each session to recover context
2. **Audit trail** — any change can be traced back to the decision that required it
3. **Blocker tracking** — blockers are recorded immediately, not discovered later during review
4. **Baseline impact tracking** — all deviations from the original spec are documented with rationale

> *See **Annex D** for full log entry examples.*

---

17. Memory and Context Management

AI sessions have finite context windows. In long projects, the AI will forget early decisions. This methodology handles this through several mechanisms.

The CLAUDE.md as persistent context

By placing instructions in CLAUDE.md, the human ensures the AI always has the operating rules even after context is compressed.

The implementation plan as state

The implementation plan contains the current state of all work. At the start of every session, the AI reads it to orient itself.

The implementation log as history

The last 2–3 log entries give the AI sufficient context to continue from where the previous session ended.

Story specs as detailed instructions

Each story spec is self-contained. The AI doesn't need to remember what was decided 10 sessions ago — it reads the story spec, which was written with all the relevant constraints already captured.

Memory system (persistent across sessions)

For the AI assistant tooling that supports it (e.g. Claude Code), a persistent memory file can be maintained at a known path. Key project facts, user preferences, and non-obvious constraints are stored there. Unlike context (which is per-session), memory persists.

The memory system is divided into types:

- **User memory** — who the user is, their expertise, preferences
- **Feedback memory** — corrections and confirmations from prior sessions
- **Project memory** — ongoing facts, decisions, deadlines

- **Reference memory** — pointers to external systems (where things are tracked, which dashboards to check)

Memory entries are indexed, not free-form. Each entry has a name, description, and type — enabling the AI to quickly determine relevance.

---

18. Common Failure Modes and How to Avoid Them

F1 — Silent architecture drift

Symptom: Code gradually stops matching the RFC as the AI makes small local decisions. **Cause:** AI optimises for "making things work" without checking the RFC on each decision. **Prevention:** Explicit RFC reference in every task spec. AI required to cite the specific RFC section it's following for each interface decision.

F2 — Test washing

Symptom: Tests exist but only test the happy path. All error cases are uncovered. **Cause:** AI writes tests to satisfy the "write tests" requirement without thinking adversarially. **Prevention:** Adversarial validation explicitly enumerates error cases. Test cases for each error path are specified in the story spec.

F3 — Completion by assertion

Symptom: AI marks task done in the log without running the tests. **Cause:** AI assumes tests would pass based on the code it wrote. **Prevention:** Log entry requires exact test runner output. "N/N tests passing" cannot be inferred — it must be observed.

F4 — Scope creep

Symptom: The AI adds "helpful" functionality beyond what the task requires. **Cause:** AI is trained to be helpful; it anticipates future needs. **Prevention:** CLAUDE.md explicitly prohibits adding features not required by the current task. The story spec has an explicit "Scope Out" section listing things that seem related but are excluded.

F5 — Context collapse

Symptom: After many sessions, the AI contradicts its earlier decisions. **Cause:** Context window compression causes earlier decisions to be forgotten. **Prevention:** Implementation plan, log, and story specs are the authoritative state — the AI reads them, not its own memory of prior sessions.

F6 — Dependency assumption

Symptom: Code is written that calls a function or uses a type that hasn't been built yet. **Cause:** AI writes "ahead" of the dependency order. **Prevention:** Pre-task checklist explicitly verifies all dependencies are satisfied. The implementation plan enforces story ordering.

F7 — Missing the integration gap

Symptom: Individual components test correctly but the integration between them is broken. **Cause:** Unit tests mock dependencies, so the interface contract between modules is only tested at integration time. **Prevention:** Integration tests are required for all critical paths. The integration checkpoint at the end of each epic is a mandatory integration test run.

---

19. Worked Example — End to End

The following is a condensed trace of a real story implementation using this methodology.

Context: Building a classification enrichment system that calls an LLM to assess qualitative business quality scores for publicly traded companies.

Step 1 — Story spec written (human)

```
STORY-040: E1–E6 Qualitative Enrichment Scores + Combined Enrichment Prompt

As a developer, I want one combined LLM call per stock returning all three ambiguous
flags
plus all six E1–E6 scores, driven by a single reviewable prompt file — so that all
qualitative classification inputs are internally consistent.

Task 1: Write combined-enrichment.md prompt body
Task 2: Implement detectCombinedEnrichment() detector
Task 3: Populate description field in enrichment sync
Task 4: Extend classificationEnrichmentSync to call detector
Task 5: Unit tests (5 cases)
```

Step 2 — Self-validation (AI, before coding)

```
SELF-VALIDATION — STORY-040

Spec match: ✓
Interface: EnrichmentScoresInput → CombinedEnrichmentOutput (RFC-007 §4.2)
Dependencies: PromptLoader (STORY-034 ✓), LLMProvider (STORY-034 ✓),
              schema columns (STORY-039 ✓)
Baseline: RFC-007 confidence gating, half-integer rounding
Tests: 5 cases per spec

Risk identified: Per-flag confidence gating — each flag has its own confidence,
but a naive implementation might use a single loop variable and shadow the
confidence name. Will use explicit destructuring to avoid.

Known constraint: description column (STORY-039) — will be null for existing stocks;
pass '' to LLM.
```

Step 3 — Adversarial validation (AI)

```
ADVERSARIAL VALIDATION — detectCombinedEnrichment

Assumption attacked: "LLM always returns a valid response"
→ What if API times out? → Function must catch all exceptions, return error
provenance.
   Test case: makeErrorProvider() → verify all 9 fields have error:true in provenance.

Assumption attacked: "All 6 scores should be null if scores_confidence < threshold"
```

```
→ This is the shared gate. Confirmed: all 6 fail together, not independently.
   The flag gate is independent (per-flag). This asymmetry must be explicit in tests.
   Test case: scoresConf=0.45 → 0 scores written, all 3 flags written (if individually
confident).

Assumption attacked: "Half-integer rounding rounds correctly"
→ Math.round(3.7 * 2) = Math.round(7.4) = 7; 7/2 = 3.5 ✓
→ Math.round(3.8 * 2) = Math.round(7.6) = 8; 8/2 = 4.0 ✓
   Both cases tested explicitly.
```

Step 4 — Implementation (AI)

AI writes the prompt file, detector module, sync service extension, and tests. Each task completed and logged before moving to the next.

Step 5 — Test run (AI, mandatory)

```
Test Suites: 43 passed, 43 total
Tests:       489 passed, 489 total
```

Step 6 — Log entry (AI)

```
## 2026-04-21 — EPIC-003.1/STORY-040: E1–E6 Qualitative Enrichment Scores

Action: Wrote combined-enrichment.md prompt; implemented detectCombinedEnrichment();
extended sync service; wrote 5 unit tests.

Result/Status: DONE □ — 489/489 unit tests passing; no new TS errors

Baseline Impact: NO
```

Step 7 — Story marked done, plan updated (AI)

Implementation plan entry for STORY-040 updated from `planned` to `done □`. EPIC-003.1 marked complete.

Annex A — CLAUDE.md Example (Full)

```
# Claude Implementation Guide for [Project Name]

## Project Overview
[One paragraph describing what is being built]

## Frozen Baseline (DO NOT MODIFY WITHOUT APPROVAL)
- **Version:** V1.0 (frozen [DATE])
- **PRD:** `/docs/prd/PRD.md`
- **RFCs:** RFC-001 through RFC-006 (accepted)
- **ADRs:** ADR-001 through ADR-011 (accepted)
```

Implementation Tracking Requirements (MANDATORY)

Before Starting Any Implementation Work

1. **Consult the implementation plan** (`/docs/architecture/IMPLEMENTATION-PLAN-V1.md`)
   - Verify current active epic/story
   - Confirm story status is `ready`
   - Check dependencies are satisfied

2. **Check for blockers** — do not start work on blocked items

3. **Link work to epic/story/task** — ALL implementation work MUST be linked. Do NOT start without this linkage.

During Implementation

1. Follow the implementation order — work epic by epic, story by story, task by task
2. Do NOT jump ahead to later epics/stories
3. Track progress continuously — update status as work progresses
4. Maintain evidence — create tests as implementation progresses

After Each Meaningful Implementation Step

1. Update the implementation log (`/docs/architecture/IMPLEMENTATION-LOG.md`)
2. Required log entry fields (all mandatory):
   - Timestamp, Epic/Story/Task ID, Action taken
   - Files changed (paths, created/modified/deleted)
   - Tests added/updated
   - Result/status
   - Blockers/issues found
   - Baseline impact (YES/NO, explain if YES)
   - Next action

Completion Requirements

Task completion requires:
- [ ] Implementation complete
- [ ] Tests added and passing
- [ ] Traceability comments in all new files
- [ ] Implementation log updated
- [ ] Task status updated to `done`

Story completion requires:
- [ ] All tasks done
- [ ] All acceptance criteria met
- [ ] Full test suite passing
- [ ] Story status updated to `done`

Prohibited Actions

- × Starting implementation without consulting implementation plan
- × Skipping implementation log updates
- × Marking work done without evidence (tests, docs, traceability)
- × Working on items not linked to epic/story/task
- × Silently changing frozen baseline assumptions

```
- × Skipping tests or regression coverage
- × Creating "temporary" code without tests
- × Jumping ahead to later epics/stories out of order

## Code Traceability Format
Every implementation file must include:
\`\`\`typescript
// EPIC-001: Platform Foundation & Deployment
// STORY-003: Provision Core GCP Infrastructure
// TASK-003-001: Create Cloud Run service configuration
\`\`\`

## Testing Standards
- Unit tests: `tests/unit/**/*.test.ts`
- Integration tests: `tests/integration/**/*.test.ts`
- Test naming: `describe('EPIC-XXX/STORY-XXX: [description]')`

## Version Control Standards
- Commit format: `[EPIC-XXX/STORY-XXX/TASK-XXX] Brief description`

**This document is authoritative. When in doubt, consult this document first.**
```

Annex B — Implementation Plan Structure (Full Example)

```
# V1 Implementation Plan

## Baseline Reference
- PRD: /docs/prd/PRD.md
- RFCs: RFC-001 through RFC-006 (accepted)
- ADRs: ADR-001 through ADR-011 (accepted)

## Status Summary
- **Current Phase:** EPIC-003 — Data Ingestion
- **Active Epic:** EPIC-003
- **Active Story:** STORY-020 — Fundamentals Sync Job
- **Overall Progress:** 2/6 epics complete (EPIC-001 □, EPIC-002 □)

## Status Model
- **planned**: Work identified, not yet validated
- **ready**: Validated against baseline, approved for implementation
- **in_progress**: Currently being worked
- **done**: Complete with evidence

## EPIC-001 — Platform Foundation & Deployment
- **Status:** done □ (2026-04-15)
- **Stories:**

### STORY-001 — Repository Bootstrap
- **Status:** done □ (2026-04-15)
- **Tasks:** TASK-001-001 □ (init repo), TASK-001-002 □ (CLAUDE.md), TASK-001-003 □
```

```
(CI)

### STORY-002 — Database Schema
- **Status:** done □ (2026-04-16)
- **Tasks:** TASK-002-001 □ (schema), TASK-002-002 □ (migration), TASK-002-003 □
(tests)

## EPIC-002 — Authentication
- **Status:** done □ (2026-04-17)
[...]

## EPIC-003 — Data Ingestion
- **Status:** in_progress
- **Stories:**

### STORY-020 — Fundamentals Sync Job
- **Status:** in_progress
- **Dependencies:** STORY-015 □ (DB schema), STORY-016 □ (FMP adapter)
- **Tasks:**
  - TASK-020-001 □ syncFundamentals() service
  - TASK-020-002 in_progress POST /api/cron/fundamentals endpoint
  - TASK-020-003 planned unit tests
```

Annex C — Story Specification Example (Full)

```
# STORY-040 — E1–E6 Qualitative Enrichment Scores + Combined Enrichment Prompt

## Epic
EPIC-003.1 — Classification LLM Enrichment

## Status
done □

## Purpose
Write the combined-enrichment.md prompt body and implement EnrichmentScoresDetector.
The combined prompt design eliminates contradiction risk from separate calls and
reduces
weekly LLM cost. All six qualitative scores (moat_strength, pricing_power,
revenue_recurrence, margin_durability, capital_intensity, qualitative_cyclicality)
feed into the Earnings Quality Scorer as optional classification inputs.

## Story
As a **developer**,
I want **one combined LLM call per stock returning all three ambiguous flags plus
all
six E1–E6 scores, driven by a single reviewable prompt file** —
so that **all qualitative classification inputs are internally consistent, STORY-
038's
combined-call architecture is fully realized, and the prompt is a single file that
can
```

be reviewed and tuned manually.

Combined Output Schema

```typescript
interface CombinedEnrichmentResult {
  holding_company: { flag: boolean; confidence: number; reason: string };
  cyclicality:     { flag: boolean; confidence: number; reason: string };
  binary_risk:     { flag: boolean; confidence: number; reason: string };
  moat_strength_score:           number;  // 1–5, half-integer steps
  pricing_power_score:           number;
  revenue_recurrence_score:      number;
  margin_durability_score:       number;
  capital_intensity_score:       number;
  qualitative_cyclicality_score: number;
  scores_confidence:  number;    // 0–1, shared across all 6 scores
  reasoning_summary:  string;    // ≤150 chars
}
```

Scope In

Task 1 — Write combined-enrichment.md prompt body

Variables: {{company_name}}, {{sector}}, {{industry}}, {{description}}, {{revenue_ttm_billions}}, {{market_cap_billions}}, {{deterministic_flags}}

Task 2 — Implement detectCombinedEnrichment()

File: src/modules/classification-enrichment/detectors/enrichment-scores.detector.ts

- Half-integer rounding: Math.round(rawScore * 2) / 2
- Scores with scores_confidence < threshold → not written; null_decision provenance
- Flags with flag.confidence < threshold → that specific flag returns null
- LLM error → returns empty object; provenance records { error: true, error_message }

Task 3 — Populate description in enrichment sync

Add description: true to DB select block. ClassificationEnrichmentInput gains description: string | null. Pass description ?? '' to LLM variables.

Task 4 — Extend classificationEnrichmentSync

Replace stub call with real detectCombinedEnrichment() call. Always call LLM (E1–E6 always require LLM even if flags are pre-determined). Single DB update per stock.

Task 5 — Unit tests (5 cases)

- All outputs above confidence threshold → all written
- scores_confidence below threshold → all 6 scores null; flags still written
- One flag below confidence → that flag null; others written
- Half-integer rounding: raw 3.7 → 3.5, raw 3.8 → 4.0
- LLM error → empty result, error in all provenance entries

Scope Out

- Live smoke test (optional Task 5 — excluded from CI)
- Description lazy-population via FMP (deferred to later story)

```
## Acceptance Criteria
- [x] One combined prompt file drives both flag and score assessment
- [x] Deterministic context passed to LLM to prevent contradictions
- [x] All 6 scores rounded to half-integer precision before DB write
- [x] Low scores_confidence sets all 6 scores to null
- [x] Individual low-confidence flags set that specific flag to null
- [x] Still one DB update per stock
- [x] Unit tests passing (489/489)

## Dependencies
- STORY-034 (LLMProvider + PromptLoader) □
- STORY-038 (sync job to extend) □
- STORY-039 (schema columns for 6 scores) □
```

Annex D — Implementation Log Entry Examples

Example 1 — Normal task completion

```
## 2026-04-21 — EPIC-003.1/STORY-040: E1–E6 Qualitative Enrichment Scores

**Epic:** EPIC-003.1 — Classification LLM Enrichment
**Story:** STORY-040
**Task:** TASK-040-001 through TASK-040-005

**Action:** Wrote production combined-enrichment.md prompt; implemented
detectCombinedEnrichment() with per-flag confidence gating, shared scores_confidence
gate, half-integer rounding, and error isolation; extended sync service to always
call combined enrichment (no needs_llm guard); updated STORY-038 tests to new
combined response format; ran full unit test suite.

**Self-validation findings:**
- Interface matches RFC-007 §4.2 exactly ✓
- Per-flag vs shared confidence asymmetry confirmed and tested ✓
- Half-integer rounding verified mathematically before implementation ✓
- Error isolation: detectCombinedEnrichment never throws — confirmed in error test ✓

**Files Changed:**
- `src/modules/classification-enrichment/prompts/combined-enrichment.md` (replaced
stub)
- `src/modules/classification-enrichment/detectors/enrichment-scores.detector.ts`
(created)
- `src/modules/classification-enrichment/jobs/classification-enrichment-
sync.service.ts` (modified)
- `tests/unit/classification-enrichment/story-040-combined-enrichment.test.ts`
(created)
- `tests/unit/classification-enrichment/story-038-classification-enrichment-
sync.test.ts` (modified)

**Tests Added/Updated:**
```

```
- `story-040-combined-enrichment.test.ts` — 5 new tests
- `story-038-*.test.ts` — updated mock format; added description field; fixed
llm_calls_made

**Result/Status:** DONE □ — 489/489 unit tests passing; 0 new TS errors

**Blockers/Issues:** None

**Baseline Impact:** NO

**Next Action:** EPIC-003.1 complete — begin EPIC-004 decomposition
```

Example 2 — Blocker encountered

```
## 2026-04-21 — EPIC-003.1/STORY-035: holding_company_flag — BC-035-001

**Epic:** EPIC-003.1
**Story:** STORY-035
**Task:** TASK-035-001

**Action:** Attempted to implement SIC code-based pre-filter for
holding_company_flag.
Discovered FMP /stable/profile endpoint does not return sic_code field.

**Baseline Impact:** YES

RFC-004 §2.1 states "FMP /profile returns sicCode for all stocks." This is incorrect
for the current FMP plan tier. The field is not present in any profile response.

**Proposed resolution:** Record as BC-035-001. Amend RFC-004. Pre-filter always
returns
null. LLM assesses holding_company_flag for all stocks.

**Status:** BLOCKED □ — awaiting human approval for baseline change

**Next Action:** Human decision on BC-035-001 resolution
```

Annex E — Self-Validation Prompt Examples

The following are example prompts used to trigger structured self-validation. These are instructions given to the AI before it begins implementation.

General self-validation prompt

```
Before writing any code for this task, perform a structured self-validation.
Work through the following checklist and write your answers explicitly.

1. SPEC MATCH: Quote the exact requirement from the story spec you are implementing.
   Confirm your plan matches it word for word, not your interpretation of it.
```

```
2. INTERFACE: State the exact input type and output type for every function you
   will write. Reference the RFC section that defines this interface.

3. DEPENDENCIES: List every function, module, or schema element your code will
   call or import. Confirm each one exists and is already implemented.

4. TEST CASES: List every test case you will write before writing any production code.
   Include: happy path, all error paths, and at least two edge cases.

5. KNOWN RISKS: State the single most likely way your implementation will be wrong.
   What assumption could be false? How will you detect it?

6. BASELINE: Does this task require any decision not already specified in an RFC or
ADR?
   If yes, stop and raise it.

Do not begin coding until this checklist is complete.
```

Interface-specific self-validation prompt

```
You are about to implement [function name]. Before writing any code:

1. Write the complete TypeScript signature including all parameter and return types.
2. Compare this signature with RFC-[N] §[section] — are they identical? If not, why?
3. List every caller of this function. What do they expect?
4. List every dependency this function calls. Are they all implemented?
5. What happens if [specific error condition]? Is that handled?
6. What happens if any input is null? Is every null case handled?

Answer each question before proceeding.
```

Annex F — Adversarial Validation Prompt Examples

General adversarial validation prompt

```
Before implementing [feature], attack your own plan. For each of the following,
describe what happens if the assumption is false, and whether the code handles it:

1. List every assumption your implementation makes about:
   - The shape of data it receives (are there fields that could be
null/undefined/wrong type?)
   - The behaviour of external systems it calls (can they time out, return empty,
return errors?)
   - The state of the database (are there columns that could be null that you assume
non-null?)
   - The execution order (is there anything that must have happened before this runs?)

2. For each assumption: what is the failure mode? Is it caught? Is it tested?
```

```
3. What would a junior developer get wrong in this implementation?

4. What edge case is most likely to be untested?

Write tests for the top 3 failure modes before writing any production code.
```

Confidence threshold adversarial prompt

```
You are implementing confidence gating for LLM outputs. Attack these assumptions:

1. The LLM always returns a value in the expected range [0, 1] for confidence.
   What if it returns 1.2? What if it returns null? What if the field is missing?

2. "Above threshold" uses >= not >. Is this correct? What happens at exactly 0.60?

3. Per-flag gating and shared score gating are different. Describe exactly how they
differ.
   Write a test that fails if you accidentally use the shared gate for flags.

4. A null result (below threshold) and an error result look different in the
provenance.
   Null: { null_decision: true }. Error: { error: true, error_message: '...' }.
   Write a test that distinguishes them.
```

Annex G — Code Traceability Examples

Service file traceability

```
// EPIC-003.1: Classification LLM Enrichment
// STORY-040: E1–E6 Qualitative Enrichment Scores + Combined Enrichment Prompt
// TASK-040-002: EnrichmentScoresDetector — single combined LLM call returning 3
flags + 6 scores
//
// Architecture: one call per stock from combined-enrichment.md prompt.
// Flags use per-flag confidence gating; scores use a single shared
scores_confidence gate.
// Half-integer rounding applied to scores before return (Math.round(v * 2) / 2).
// LLM errors are caught internally — function never throws.
// RFC-007: LLMProvider interface, confidence gating, provenance shape

export async function detectCombinedEnrichment(
  stock: EnrichmentScoresInput,
  llmProvider: LLMProvider,
  promptLoader: PromptLoader,
): Promise<CombinedEnrichmentOutput> { ... }
```

Non-obvious constraint comment

```
// BC-035-001: FMP /stable/profile does not return SIC codes in the current plan
tier.
// holding_company_flag pre-filter always returns null in production.
// All stocks proceed to LLM assessment for this flag.
sicCode: null,
```

Ordering dependency comment

```
// Must run AFTER syncFundamentals() and syncMarketCapAndMultiples().
// Reads marketCap from the stocks table to compute forward EV/EBIT and EV/Sales.
// If marketCap is null (market-cap sync not yet run), EV multiples will be null.
export async function syncForwardEstimates(...) { ... }
```

Test traceability

```
// EPIC-003.1/STORY-040: detectCombinedEnrichment
// TASK-040-004: Unit tests

describe('EPIC-003.1/STORY-040: detectCombinedEnrichment', () => {
  it('all outputs above threshold → all 3 flags written, all 6 scores written', ...)
  it('scores_confidence below threshold → all 6 scores empty, flags still written',
...)
  it('one flag below confidence threshold → that flag null, others written', ...)
  it('half-integer rounding: raw 3.7 → 3.5, raw 3.8 → 4.0', ...)
  it('LLM API error → empty flags and scores, error in provenance for all fields',
...)
})
```

Annex H — RFC and ADR Templates

RFC Template

```
# RFC-NNN — [Title]

**Status:** draft | in_review | accepted | superseded
**Date:** YYYY-MM-DD
**Author:** [Name]
**Supersedes:** RFC-NNN (if applicable)
**Superseded by:** RFC-NNN (if applicable)

## Problem Statement
[What architectural question does this RFC answer? Why does it need to be decided?]

## Context
[Background information necessary to understand the decision]

## Options Considered
```

```
### Option A — [Name]
[Description]
**Pros:** [...]
**Cons:** [...]

### Option B — [Name]
[Description]
**Pros:** [...]
**Cons:** [...]

## Decision
**We choose Option [X].**

[Rationale — why this option over the others]

## Consequences
**Positive:** [What becomes easier or better]
**Negative:** [What becomes harder or is constrained]
**Neutral:** [What changes but is neither better nor worse]

## Implementation Notes
[Specific guidance for implementers — types, file locations, patterns to follow]
```

ADR Template

```
# ADR-NNN — [Title]

**Status:** accepted | superseded
**Date:** YYYY-MM-DD
**Context:** RFC-NNN §[section] (the RFC this ADR elaborates on)

## Decision
[One sentence stating the decision]

## Rationale
[2–4 sentences explaining why]

## Consequences
[What this decision requires or prevents in implementation]
```

This document may be freely shared and reproduced. The methodology described here is the result of practical application in production software development and is offered as a practitioner's guide, not a theoretical framework.